Dominando el Flujo y la Memoria en Java: De la Lógica a la Estructura Modular

PROGRAMACIÓN II — UTN

En esta microclase aprenderemos a controlar el flujo de ejecución, gestionar la memoria, modularizar código y aplicar buenas prácticas profesionales en Java.

🎯 Objetivo 1

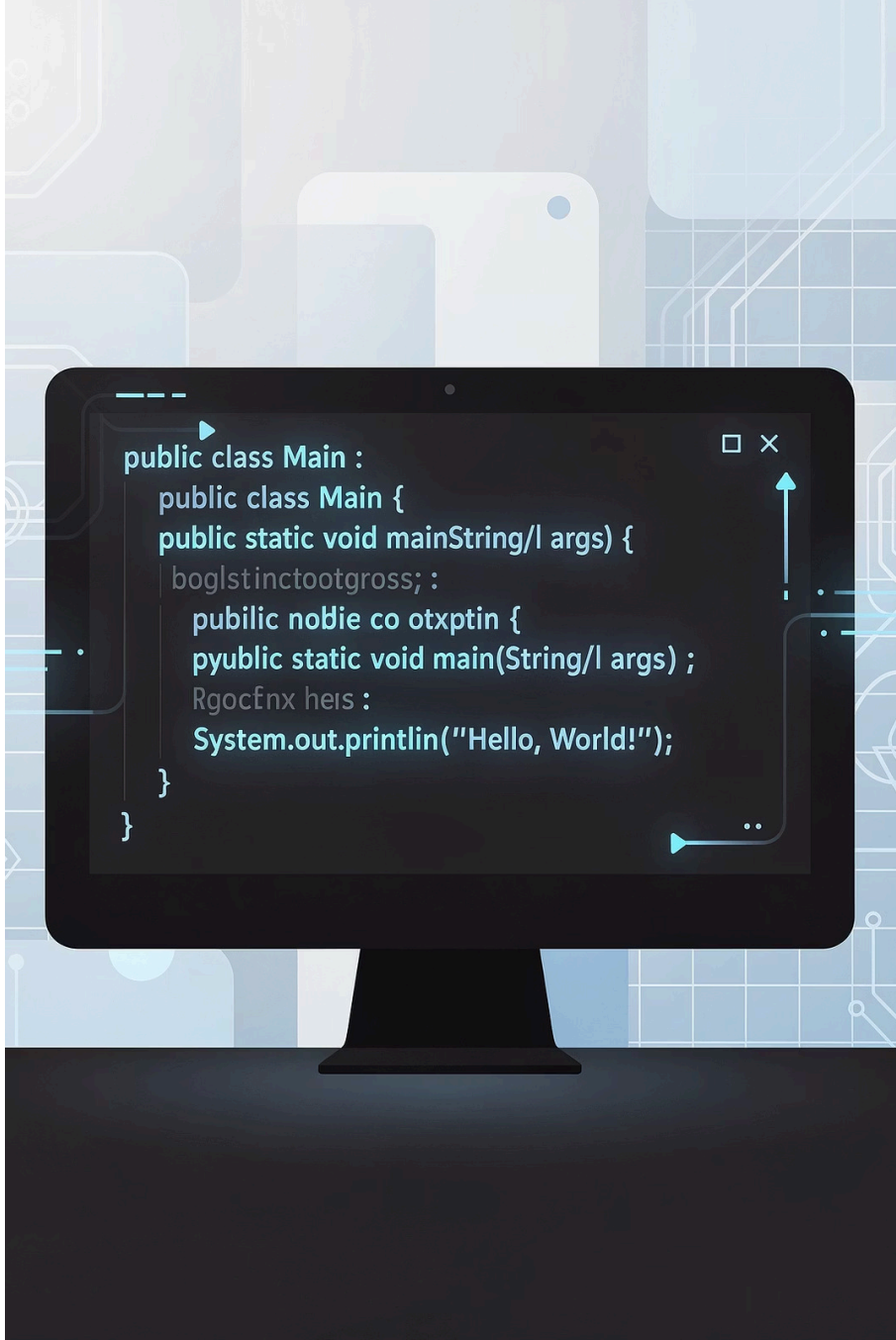
Dominar estructuras de control: condicionales y bucles

🎯 Objetivo 2

Comprender métodos, scope y recursividad

🎯 Objetivo 3

Gestionar memoria y estructuras de datos en Java



Lógica de Comparación

Los **operadores relacionales** (`==`, `!=`, `<`, `>`, `<=`, `>=`) comparan valores primitivos. Sin embargo, con objetos y Strings, `==` compara **referencias en memoria**, no el contenido.

❌ Incorrecto con Strings

```
String a = new String("hola");
String b = new String("hola");
// a == b → false (distintas refs)
```

✅ Correcto con .equals()

```
String a = new String("hola");
String b = new String("hola");
// a.equals(b) → true (mismo contenido)
```

Regla de oro: usa `==` para primitivos (`int`, `boolean`, etc.) y `.equals()` para objetos y Strings.

Toma de Decisiones

Java ofrece tres mecanismos principales para bifurcar la ejecución según condiciones. Elegir el correcto mejora la legibilidad y el rendimiento del código.

1

if-else

Para condiciones booleanas complejas o rangos de valores.

```
if (nota >= 6) {
  System.out.println("Aprobado");
} else {
  System.out.println("Reprobado");
}
```

2

switch

Para múltiples valores discretos de una misma variable.

```
switch (dia) {
  case 1: System.out.println("Lunes");
break;
  case 2: System.out.println("Martes");
break;
  default: System.out.println("Otro");
}
```

3

Operador Ternario

Para asignaciones compactas en una sola línea.

```
String res = (nota >= 6) ? "Aprobado" :
"Reprobado";
```

Errores Comunes de Sintaxis

Los errores de sintaxis más frecuentes en Java involucran operadores compuestos y la confusión entre asignación y comparación. Identificarlos temprano ahorra horas de depuración.

❌ Errores frecuentes

```
// Espacio incorrecto en operador compuesto
total + = 10;  // Error de compilación

// Asignación en lugar de comparación
if (x = 5) {   // Error: asigna, no compara
  ...
}
```

✅ Forma correcta

```
// Sin espacio en operador compuesto
total += 10;   // Correcto

// Comparación con ==
if (x == 5) {  // Correcto
  ...
}
```

→ `+=`, `-=`, `*=`, `/=` son operadores compuestos: **nunca llevan espacio** entre los símbolos.

→ `=` asigna un valor; `==` compara dos valores. Confundirlos es uno de los bugs más comunes.

Programación Defensiva

La programación defensiva busca **evitar cierres inesperados** (crashes) anticipando condiciones de error antes de que ocurran. Un programa robusto valida sus entradas y maneja casos límite.

Validar antes de operar

Verificar que un objeto no sea `null` antes de llamar sus métodos. Verificar rangos antes de dividir o acceder a arreglos.

Aprovechar NetBeans

El IDE subraya en amarillo y rojo los posibles errores. Las sugerencias (*hints*) del editor son tu primer debugger: léelas siempre.

Manejo de excepciones

Usar bloques `try-catch` para capturar errores en tiempo de ejecución y ofrecer mensajes claros al usuario en lugar de un crash.

El Arte de Repetir: Bucles en Java

Los bucles permiten ejecutar un bloque de código múltiples veces. La elección del tipo correcto depende de **cuándo se conoce la condición de parada**.

while

Evalúa la condición **antes** de ejecutar. Úsalo cuando no sabes cuántas iteraciones habrá.

```
while (x < 10) { x++; }
```

for

Ideal cuando conoces el **número exacto** de iteraciones. Inicialización, condición e incremento en una línea.

```
for (int i=0; i<10; i++) { ... }
```

do-while

Ejecuta el bloque **al menos una vez**, luego evalúa la condición. Útil para menús interactivos.

```
do { leerOpcion(); } while (op != 0);
```

Herramientas de Control: Contadores y Acumuladores

Dentro de los ciclos, los **contadores** y **acumuladores** son variables auxiliares esenciales para registrar cuántas veces ocurre algo o para sumar valores progresivamente.

Contador

Incrementa de a uno (o un valor fijo) en cada iteración. Registra **cuántas veces** ocurre una condición.

```
int contador = 0;
for (int i = 0; i < 100; i++) {
  if (i % 2 == 0) {
    contador++; // cuenta pares
  }
}
```

Acumulador

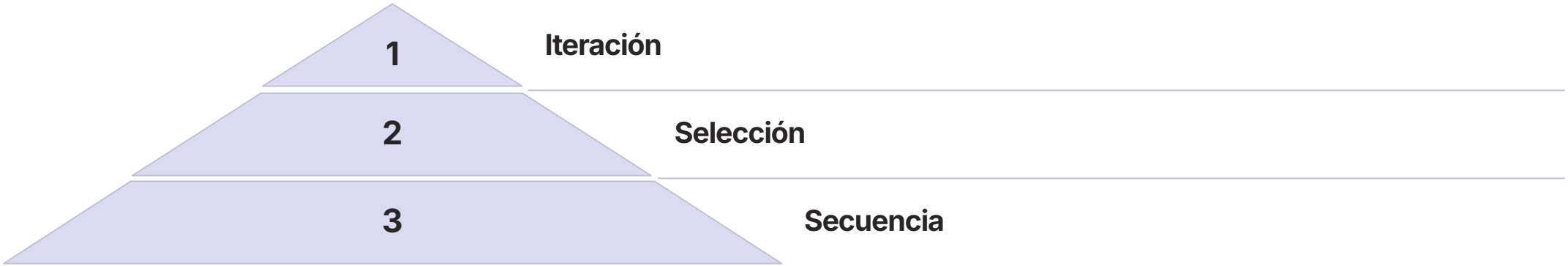
Suma valores variables en cada iteración. Calcula **totales, promedios** o cualquier agregación.

```
int suma = 0;
for (int i = 1; i <= 10; i++) {
  suma += i; // acumula 1+2+...+10
}
// suma = 55
```

Siempre inicializa contadores en `0` y acumuladores en `0` (o el elemento neutro de la operación) **antes** del bucle.

Teorema del Programa Estructurado

Cualquier algoritmo computable puede expresarse usando únicamente **tres estructuras de control**. Este principio, demostrado por Böhm y Jacopini, es la base de toda la programación imperativa.



Secuencia

Las instrucciones se ejecutan una tras otra en orden lineal.

Selección

Se elige un camino u otro según una condición (`if`, `switch`).

Iteración

Un bloque se repite mientras se cumpla una condición (`while`, `for`).

Ciclos Anidados: Estructuras Matriciales

Un bucle dentro de otro permite recorrer estructuras bidimensionales como matrices o generar patrones visuales en consola. El bucle externo controla las **filas** y el interno las **columnas**.

Código: Tabla de multiplicar

```
for (int f = 1; f <= 3; f++) {
  for (int c = 1; c <= 3; c++) {
    System.out.print(f * c + "\t");
  }
  System.out.println();
}
// Salida:
// 1  2  3
// 2  4  6
// 3  6  9
```

Código: Patrón de asteriscos

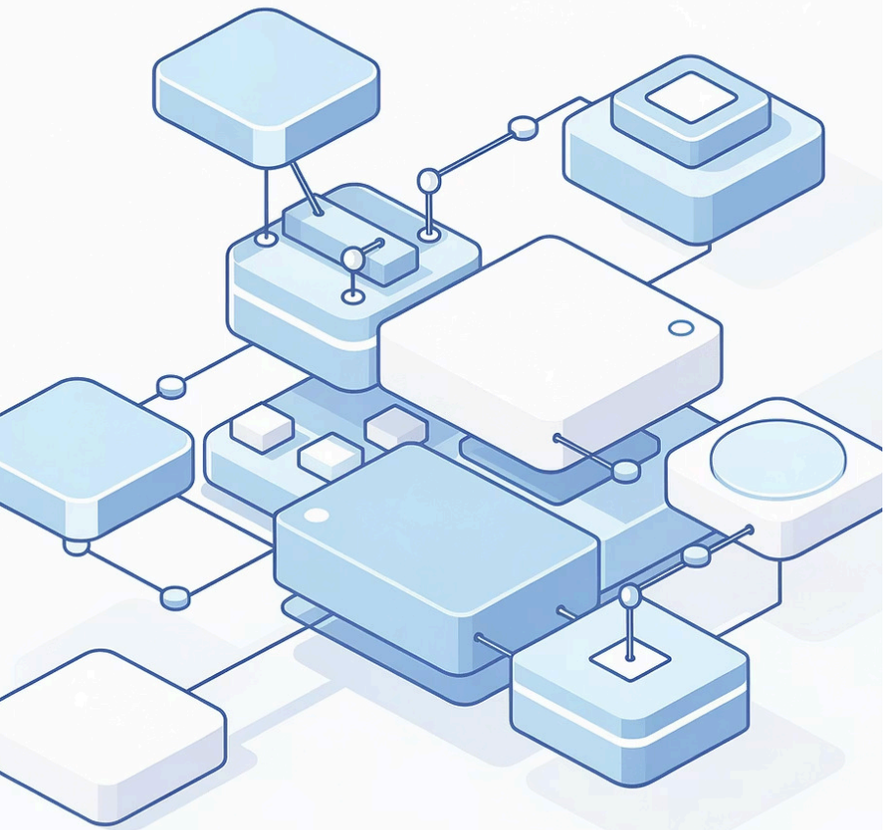
```
for (int f = 1; f <= 4; f++) {
  for (int c = 1; c <= f; c++) {
    System.out.print("* ");
  }
  System.out.println();
}
// Salida:
// *
// * *
// * * *
// * * * *
```

Regla clave: por cada iteración del bucle externo, el bucle interno se ejecuta **completo**. Visualiza esto como recorrer fila por fila una hoja cuadriculada.

Debugging de Bucles

Los **bucles infinitos** son uno de los errores más comunes. Ocurren cuando la condición de salida nunca se vuelve falsa. El debugger de NetBeans permite ejecutar el programa en "cámara lenta".

- **Auditar la condición:** Verificar que la variable de control se modifique dentro del bucle y que la condición eventualmente sea falsa.
- **Breakpoints:** Colocar un punto de parada (*breakpoint*) en la línea del bucle con `F8` para ejecutar paso a paso.
- **Watch variables:** Observar el valor de las variables en cada iteración desde la ventana "Variables" del debugger.
- **Forzar detención:** Si el programa se cuelga, usar el botón "Stop" en NetBeans y revisar la lógica de la condición de salida.



Modularización: ¿Qué es un Método?

Un **método** es un bloque de código con nombre que realiza una tarea específica y puede ser invocado desde cualquier parte del programa. La modularización es el arte de dividir un problema grande en piezas pequeñas y manejables.

Elimina duplicación

Escribe la lógica una sola vez y reutilízala cuantas veces necesites con una simple llamada al método.

Facilita el mantenimiento

Si hay un error, lo corriges en un solo lugar. El código modular es más fácil de leer, probar y modificar.

Trabajo en equipo

Distintos desarrolladores pueden trabajar en distintos métodos simultáneamente sin interferir entre sí.

Anatomía de un Método en Java

Todo método en Java tiene una firma que define su comportamiento, visibilidad y lo que devuelve. Comprender cada parte es fundamental para diseñar métodos correctamente.

```
// modificadorAcceso  tipoRetorno  nombre  (parámetros)
  public            static int   sumar   (int a, int b) {
     return a + b;  // valor de retorno
  }

// Método sin retorno (void)
  public static void saludar(String nombre) {
     System.out.println("Hola, " + nombre);
     // no lleva return (o return; vacío)
  }
```

Modificador

`public` / `private`: define quién puede llamar al método.

Tipo de retorno

`int`, `String`, `void`: qué tipo de dato devuelve (o nada).

Parámetros

Variables de entrada que recibe el método al ser invocado.

Alcance (Scope) y Sobrecarga de Métodos

Scope: Variables Locales vs. de Clase

Una **variable local** existe solo dentro del método donde fue declarada. Una **variable de clase** (atributo) es accesible desde todos los métodos de la clase.

```
class Ejemplo {
  static int contador = 0; // de clase

  static void incrementar() {
    int temp = 1; // local: solo aquí
    contador += temp;
  }
}
```

Sobrecarga (Overloading)

Java permite definir **múltiples métodos con el mismo nombre** siempre que difieran en el número o tipo de parámetros. El compilador elige cuál usar según los argumentos.

```
static int sumar(int a, int b) {
 return a + b;
}
static double sumar(double a, double b) {
 return a + b;
}
// sumar(2, 3) → usa el primero
// sumar(2.5, 3.1) → usa el segundo
```

Diseño Modular Profesional

Un código profesional no solo funciona: es legible, mantenible y escalable. Dos principios guían el diseño de métodos de calidad.

Principio de Responsabilidad Única (SRP)

Cada método debe hacer **una sola cosa y hacerla bien**. Si un método calcula, valida Y muestra resultados, debe dividirse en tres métodos separados. Un buen nombre de método describe completamente su única responsabilidad.

Regla de las 20 líneas

Un método no debería superar las **20 líneas de código**. Si lo hace, es una señal de que está haciendo demasiado y debe refactorizarse. Métodos cortos son más fáciles de leer, probar y depurar.

Nombres autoexplicativos

Prefiere `calcularPromedio()` sobre `calc()`. El nombre debe revelar la intención. Un código bien nombrado **se documenta solo** y reduce la necesidad de comentarios.

Documentación con Javadoc

Javadoc genera documentación HTML automáticamente a partir de comentarios especiales en el código. Documentar es escribir para **humanos**, no para la máquina.

Sintaxis Javadoc

```
/**
 * Calcula el promedio de un arreglo.
 * @param valores arreglo de enteros
 * @return promedio como double
 * @author Docente UTN
 */
public static double calcularPromedio(int[] valores) {
    // implementación...
}
```

Configuración en NetBeans

Por defecto, NetBeans solo documenta métodos `public`. Para incluir métodos `private`:

1. Ir a **Tools → Options → Java**
2. Seleccionar la pestaña **Javadoc**
3. Activar la opción **"Show private members"**
4. Generar con **Run → Generate Javadoc**

Recursividad: La Función que se Llama a Sí Misma

Un método es **recursivo** cuando se invoca a sí mismo para resolver una versión más pequeña del mismo problema. Es elegante pero requiere disciplina: sin un **caso base**, produce un bucle infinito.

Ejemplo: Factorial

```
static int factorial(int n) {
  // CASO BASE: condición de parada
  if (n == 0) return 1;

  // CASO RECURSIVO: se llama a sí mismo
  return n * factorial(n - 1);
}
// factorial(4) = 4 * 3 * 2 * 1 = 24
```

Las dos partes obligatorias

1 **Caso Base**

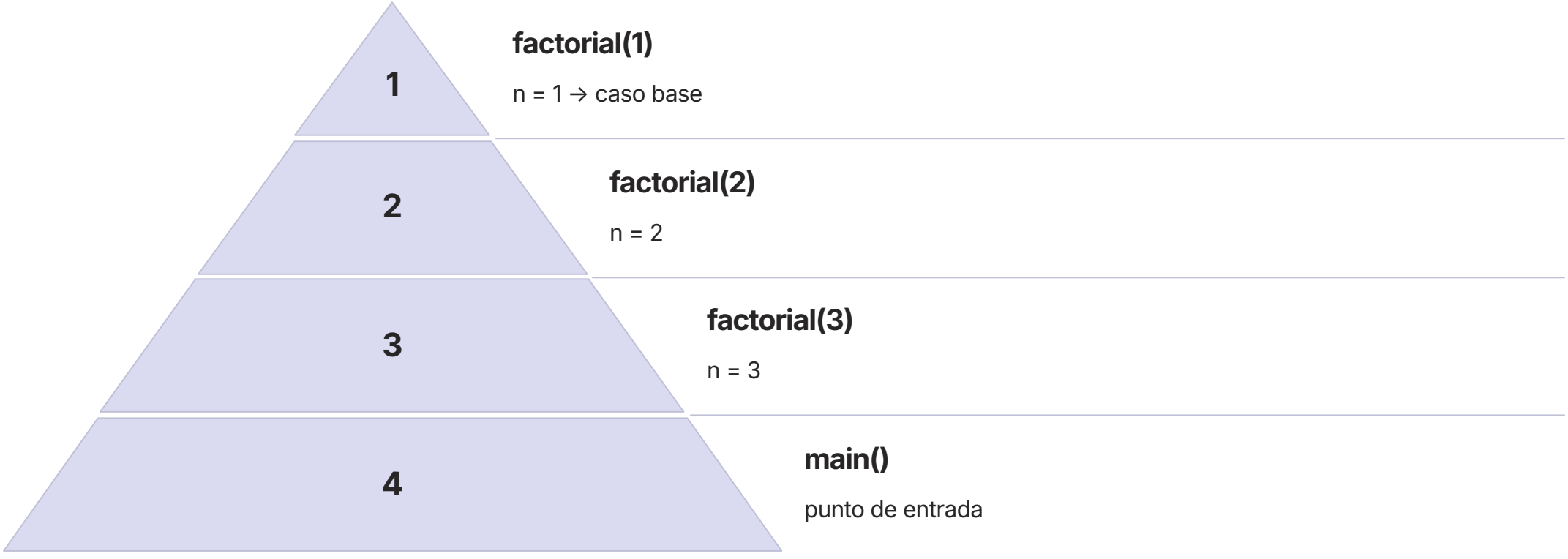
La condición que **detiene** la recursión. Sin él, el programa colapsa con `StackOverflowError`.

2 **Caso Recursivo**

La llamada al mismo método con un argumento que **se acerca** al caso base en cada paso.

Gestión de Memoria: El Stack (Pila)

Cada vez que se llama a un método, Java reserva un bloque de memoria llamado **Stack Frame** en la Pila (Stack) de la RAM. Al finalizar el método, ese frame se libera automáticamente.



La pila crece hacia arriba con cada llamada y se contrae al retornar. Visualízala como una **pila de platos**: el último en entrar es el primero en salir (LIFO). Cada frame almacena las variables locales y el punto de retorno del método.

Arreglos en Java: Declaración e Inicialización

En Java, los arreglos (arrays) son una estructura de datos fundamental utilizada para almacenar una colección de elementos del mismo tipo. Una característica clave es que su tamaño es fijo y se define en el momento de su creación.

```
// Tamaño definido con new
int[] numeros = new int[5];
String[] nombres = new String[3];

// Inicialización con valores directos
double[] precios = {19.99, 25.50, 9.75};
```

Valores por Defecto

Numéricos

(`int`, `double`, etc.) → `0`

Booleanos

(`boolean`) → `false`

Tipos de Objeto

(`String`, clases, etc.) → `null`

Arreglos en Java: Acceso, Modificación y Recorrido

Los índices siempre inician en 0. Usa .length para evitar errores al recorrer el arreglo.

Acceso y Modificación

```
int[] edades = {25, 30, 22};

// Acceder al segundo elemento
int segunda = edades[1]; // → 30

// Modificar el primer elemento
edades[0] = 26; // → {26, 30, 22}

// Propiedad .length
int total = edades.length; // → 3
```

Recorrido con for

```
int[] puntuaciones = {85, 92, 78, 95, 88};

for (int i = 0; i < puntuaciones.length; i++) {
  System.out.println("Índice " + i + ": " + puntuaciones[i]);
}
```

Siempre usa `arr.length` como límite en el bucle `for` para evitar `ArrayIndexOutOfBoundsException`.

Errores Críticos de Memoria

Dos errores de memoria son especialmente frecuentes en Java y es fundamental distinguirlos para depurarlos rápidamente.

StackOverflowError

Causa: Recursión infinita — el caso base nunca se alcanza y la pila se llena hasta desbordarse.

```
static int error(int n) {
  // ¡Sin caso base!
  return n * error(n - 1);
  // → StackOverflowError
}
```

Solución: Verificar que el caso base exista y sea alcanzable.

ArrayIndexOutOfBoundsException

Causa: Acceder a un índice que no existe en el arreglo (menor que 0 o mayor/igual al tamaño).

```
int[] arr = new int[3]; // índices: 0,1,2
arr[3] = 10;
// → ArrayIndexOutOfBoundsException
```

Solución: Usar `arr.length` como límite en los bucles.

Conclusión y Checklist de Buenas Prácticas

Un programador profesional no solo escribe código que funciona: escribe código que **otros pueden leer, mantener y extender**. Estas prácticas resumen todo lo aprendido en la microclase.

main() corto

El método `main` debe solo orquestar llamadas a métodos, no contener lógica de negocio.

Nombres claros

Variables, métodos y clases con nombres descriptivos que revelen su intención sin necesidad de comentarios.

Código autoexplicativo

Prefiere claridad sobre brevedad. Un código legible es más valioso que uno "inteligente" pero críptico.

Un método, una tarea

Aplica el SRP: métodos cortos (<20 líneas), con una sola responsabilidad y bien documentados con Javadoc.

> "El código se escribe una vez pero se lee cien veces. Escríbelo para quien lo leerá mañana."